

DAO GOVERNANCE ARCHITECTURE

Layer 4 Integration Gates Specification

Self-Evolving & Self-Optimizing Quadratic Voting Module

Data & Oracle Layer — Agentic Coding IDE Integration Gates

Generated: 2026-05-06

Executive Summary

This document specifies the Integration Gates for Layer 4 (Data & Oracle Layer) of a Self-Evolving DAO Governance Architecture based on Quadratic Voting. These gates are designed for enforcement within agentic coding IDEs to ensure that all generated code for the oracle infrastructure adheres to the foundational principles of cryptographic verifiability, decentralization, economic security, and strict cross-layer isolation.

ARCHITECTURAL CONTEXT: Layer 4 serves as 'The Senses' of the DAO—feeding verified, tamper-proof data into Layer 3 (The Intelligence/Optimization Layer) so it can accurately calculate parameter adjustments. Compromise at this layer results in poisoned optimization, making these gates critical for systemic safety.

Foundational Principles

Before implementing any Layer 4 component, the agentic IDE must enforce the following first-principles derived from the DAO architecture:

The Sybil Principle

Quadratic voting mathematically collapses if one user can split their identity. Ten votes cost 100 credits for one identity, but only 10 credits if split across ten identities. Absolute Sybil resistance is the foundational dependency. Layer 4 must not introduce identity-fragmentation vectors through its data feeds.

The Optimization Principle

A static rule applied to a dynamic environment eventually fails. Parameters (quorum, voting duration, credit allocation) must be treated as variables in a continuous optimization function, not as constants. Layer 4 must provide the real-time data necessary for this continuous optimization.

The Safety Principle

Autonomous evolution introduces systemic risk. Any automated parameter adjustment must be mathematically bounded to prevent runaway feedback loops or malicious capture. Layer 4 data must include confidence and freshness metrics to enable these safety bounds.

Gate L4-01: Cryptographic Verifiability Gate

PURPOSE: Every data point entering Layer 3 must carry a cryptographically verifiable proof of authenticity.

Pre-conditions

- Oracle response must include a digital signature from the oracle node.
- Signature must be recoverable to a known, whitelisted oracle operator address.
- Data payload must include a block height or timestamp attestation.

Post-conditions

- Layer 3 receives only **VerifiedDataPacket** structs, never raw primitives.
- Failed signature verification **MUST** reject the entire packet, never partial data.

Agent Invariant Check

```
# Agent must enforce this pattern
assert data_packet.signature is not None
assert recover_signer(data_packet.signature) in WHITELISTED_ORACLES
assert data_packet.block_height <= current_block # No future-dated data
```

FAILURE MODE: If signature missing or invalid → discard packet, increment *oracle_fault_counter*, trigger aggregator fallback.

Gate L4-02: Multi-Source Aggregation Gate

PURPOSE: Prevent single-source manipulation attacks against the optimization engine.

Pre-conditions

- Minimum **N=3** independent oracle sources for any metric feeding Layer 3.
- Sources must be geographically and operator-diverse (no shared infrastructure).
- Aggregation function must be configurable: *median* (default), *trimmed_mean*, or *consensus_threshold*.

Post-conditions

- Layer 3 receives a single **AggregatedValue** with attached *confidence_score* and *deviation_index*.
- If source deviation $> 2\sigma$ from aggregate, flag source as **SUSPECT**.

Agent Constraint

```
# Hard constraint for agent
MIN_ORACLE_SOURCES = 3
AGGREGATION_FUNCTION = "median" # Resistant to outlier manipulation

assert len(sources) >= MIN_ORACLE_SOURCES
assert len(set(s.operator for s in sources)) >= MIN_ORACLE_SOURCES # No Sybil sources
```

Forbidden Patterns

- Direct passthrough of single oracle value to Layer 3.
- Averaging without outlier detection.
- Allowing operator to run multiple nodes counted as 'independent'.

Gate L4-03: Oracle Network Decentralization Gate

PURPOSE: Ensure the oracle mechanism itself is decentralized, not just multi-source.

Pre-conditions

- On-chain registry of active oracle nodes with stake-weighted reputation.
- Node selection for aggregation must use a deterministic, verifiable random function (VRF).
- Stake slashing condition defined for nodes providing out-of-bounds data.

Post-conditions

- Any aggregation round uses a pseudorandom subset of available nodes.
- Node reputation scores decay if flagged by Gate L4-02.

Agent Implementation Rule

```
// Agent must generate code following this pattern
function selectOracleNodes(bytes32 seed, uint256 count) internal view returns (address[] memory) {
    require(count >= MIN_ORACLE_SOURCES, "Insufficient nodes");
    // VRF-based selection from staked node pool
    return vrfSample(STAKED_NODE_REGISTRY, seed, count);
}
```

Gate L4-04: Data Freshness & Latency Gate

PURPOSE: Stale data is attackable data. Governance decisions require timely inputs.

Pre-conditions

Each data feed defines **MAX_DATA_AGE** per metric type:

- Token price: **300 seconds**
- Treasury velocity: **3600 seconds**
- Voter turnout: **86400 seconds**
- Oracle response must include *reportedAt* timestamp.

Post-conditions

- Data older than **MAX_DATA_AGE** is rejected before aggregation.
- Layer 3 receives *freshness_score* (0-1) alongside each aggregated value.

Agent Validation

```
def validate_freshness(packet, metric_type):
    max_age = FRESHNESS_RULES[metric_type]
    age = current_time - packet.reported_at
    if age > max_age:
        raise StaleDataRejection(f"Data stale by {age}s")
    return 1.0 - (age / max_age) # freshness_score
```

Gate L4-05: Cross-Layer Interface Contract Gate

PURPOSE: Maintain strict boundaries. Layer 4 feeds Layer 3; Layer 4 never touches Layer 2 directly.

Pre-conditions

- Layer 4 output schema is versioned and immutable during a governance epoch.
- Layer 3 consumes only through a defined **IDataOracle** interface.
- No Layer 4 function may call Layer 2 (State/Execution) contract addresses.

Interface Contract (Agent Must Generate)

```
interface IDataOracle {
    struct AggregatedMetric {
        bytes32 metricId;
        uint256 value;
        uint256 confidenceScore;    // 0-10000 (basis points)
        uint256 freshnessScore;    // 0-10000
        uint256 blockHeight;
        bytes32 aggregationProof;  // Merkle root of source attestations
        uint256 sourceCount;
    }

    function getMetric(bytes32 metricId) external view returns (AggregatedMetric memory);
    function getMetricBatch(bytes32[] calldata metricIds) external view returns (AggregatedMetric[] memory);
}
```

Forbidden Patterns

- Layer 4 writing to Layer 2 state.
- Layer 4 emitting events that Layer 2 listens to directly.
- Layer 4 accessing treasury balances or execution logic.

Gate L4-06: Failover & Graceful Degradation Gate

PURPOSE: If oracle infrastructure fails, the system must degrade safely, not catastrophically.

Pre-conditions

- Define **DEGRADED_MODE_THRESHOLD**: if $< 60\%$ of expected sources respond, enter degraded mode.
- Define **SAFE_STATIC_FALLBACK**: last known good values cached on-chain with exponential decay.

Post-conditions

- In degraded mode: Layer 3 receives *confidenceScore* = 0, triggering conservative parameter adjustments only.
- If degraded mode persists > 3 epochs, Layer 5 (Failsafe) timelock extends automatically.

Agent Logic

```
def assess_system_health(responses, expected):  
    response_rate = len(responses) / expected  
    if response_rate < DEGRADED_MODE_THRESHOLD:  
        return Mode.DEGRADED, cache.get_last_known_good()  
    return Mode.NORMAL, aggregate(responses)
```

Gate L4-07: Economic Security & Incentive Alignment Gate

PURPOSE: Oracle operators must have more to lose from attack than to gain.

Pre-conditions

- Minimum stake per oracle node: **STAKE_MIN = 10_000 * 10**18** (governance token units).
- Slashing condition: Node flagged by L4-02 three times within 10 epochs → 50% stake slashed.
- Reward condition: Accurate nodes within 1σ of aggregate receive reward from DAO treasury.

Post-conditions

- Stake contract is non-upgradeable (mirrors Layer 2 constraint).
- Slashing requires multi-sig from Layer 5 veto council (prevents griefing).

Agent Must Enforce

```
modifier sufficientStake(address node) {  
    require(stakeContract.getStake(node) >= STAKE_MIN, "Insufficient stake");  
    _;  
}
```

Gate L4-08: Quantifiable Off-Chain Data Sanitization Gate

PURPOSE: Off-chain data (social sentiment, GitHub activity, etc.) is high-risk.

Pre-conditions

- Off-chain data must pass through a **verifiable computation** layer (e.g., zkTLS, TEE attestation, or ZK-proof of API response).
- Raw off-chain data never touches Layer 3 directly—only attested, transformed outputs.

Post-conditions

- Off-chain metrics carry *attestationType* (ZK_PROOF | TEE_QUOTE | MULTISIG_ORACLE).
- Layer 3 weights off-chain metrics lower (max **20%** influence on any optimization).

Agent Constraint

```
OFF_CHAIN_MAX_WEIGHT = 0.20 # 20% cap on off-chain data influence

if metric.source_type == SourceType.OFF_CHAIN:
    assert metric.attestation is not None
    final_weight = min(proposed_weight, OFF_CHAIN_MAX_WEIGHT)
```

Agentic IDE Prompt Injection

Copy-paste the following into your agentic IDE's rules/gates file (e.g., *.cursorrules*, *windsurf-gates.md*):

```
## LAYER 4 ORACLE INTEGRATION GATES (ENFORCED)
```

Before generating ANY code for Layer 4 (Data & Oracle Layer), you MUST:

1. Verify all data packets include cryptographic signatures (L4-01). Never pass unsigned data to Layer 3.
2. Enforce minimum 3 independent oracle sources (L4-02) with median aggregation. Never implement single-source.
3. Use VRF-based node selection from a staked registry (L4-03). Never hardcode oracle addresses.
4. Reject stale data exceeding metric-specific MAX_DATA_AGE (L4-04).
5. Implement only the IDataOracle interface for Layer 3 consumption (L4-05). NEVER write to Layer 2 state or
6. Implement degraded mode fallback when response rate < 60% (L4-06).
7. Require economic stake and slashing conditions for all oracle nodes (L4-07).
8. Cap off-chain data influence at 20% and require verifiable attestations (L4-08).

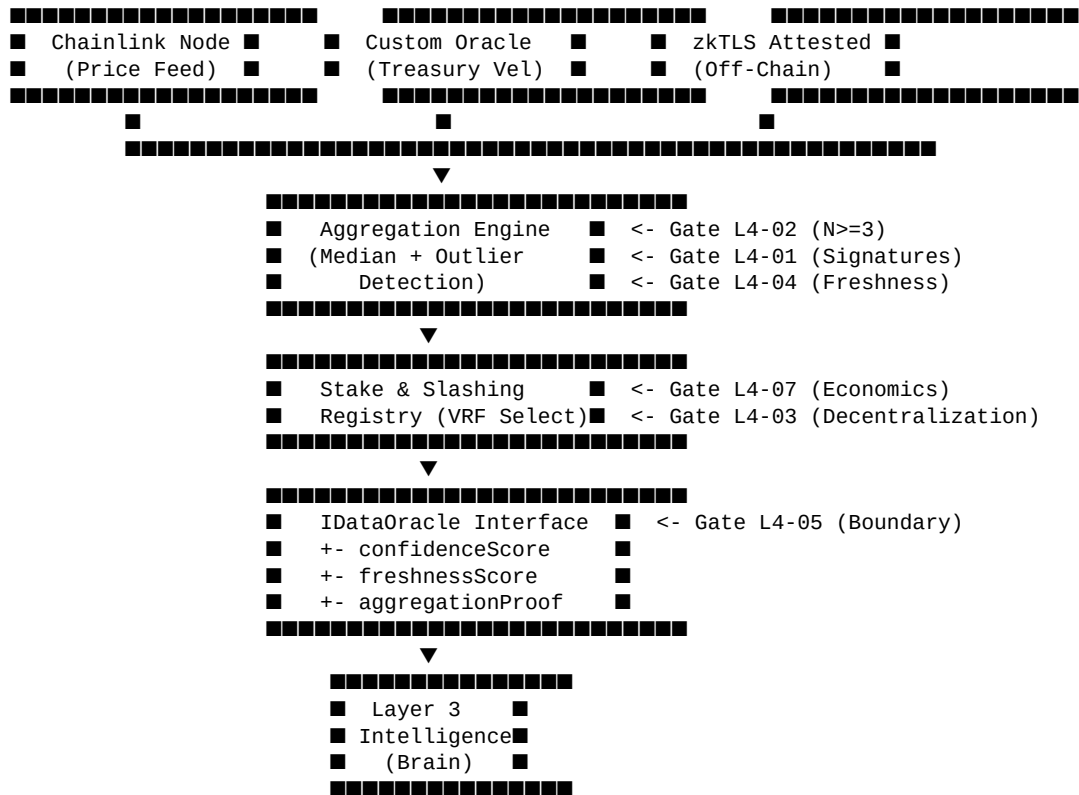
```
### INVARIANTS (Never Violate):
```

- Layer 4 NEVER touches treasury funds.
- Layer 4 NEVER executes proposals.
- Layer 4 NEVER modifies Layer 2 state.
- All outputs to Layer 3 MUST include confidenceScore and freshnessScore.

If any requirement is impossible to satisfy, HALT and request human clarification rather than generating non-

Quick Reference: Layer 4 Data Flow

Layer 4 Data Flow Diagram (Text Representation):



These gates ensure your agentic IDE generates Layer 4 code that is **manipulation-resistant**, **cryptographically verifiable**, and **strictly bounded**—preventing the optimization engine from being poisoned at its sensory layer.

Appendix A: DAO Architecture Context

The following summarizes the original DAO Governance Architecture from which these gates are derived:

Phase 1: First Principles Derivation

- **The Sybil Principle:** Quadratic voting collapses without 1 Identity = 1 Verified Entity.
- **The Optimization Principle:** Static rules fail in dynamic environments; parameters must be variables.
- **The Safety Principle:** Automated adjustments must be mathematically bounded.

Layer 1: Identity & Sybil-Resistance

Zero-Knowledge Proofs tied to decentralized identifiers. Must maintain absolute voter anonymity.

Layer 2: State & Execution Layer

Immutable smart contracts handling core quadratic logic. Cost = $\Sigma(v_i^2)$. Core logic is non-upgradable.

Layer 3: Intelligence & Optimization Layer

Applied AI/ML analyzing historical voting data. Optimizes for KPIs (voter apathy reduction, proposal success). Cannot touch treasury funds or force-execute proposals. Adjusts only predefined variables within hardcoded bounds.

Layer 5: Failsafe / Veto Layer

Timelock delay on all parameter adjustments. Ragequit function or Supreme Veto by distributed multisig. Veto can only cancel automated adjustments, not initiate actions. Veto triggers revert to safe static baseline.

The Self-Evolving Loop (OODA)

- **ACT:** Users submit proposals and vote via Layer 2.
- **OBSERVE:** Layer 4 aggregates data on turnout, distribution, treasury impact.
- **ORIENT:** Layer 3 processes data, identifies inefficiencies.
- **ADAPT:** Layer 3 formulates parameter adjustment and broadcasts to Layer 2.
- **VERIFY:** Adjustment sits in Layer 5 timelock. If untouched by veto, executes for next epoch.